

ML-KEM (Crystal-Kyber)

1. Time complexity:

- The time complexity of ML-KEM's core operations is dominated by polynomial arithmetic in the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ and operations on a $k \times k$ matrix of such polynomials.
 - Number-Theoretic Transform (NTT): ML-KEM uses the NTT to drastically speed up polynomial multiplication. A standard polynomial multiplication would take $O(n^2)$ time, but the NTT reduces this to $O(n \log n)$.
 - Key Generation, Encapsulation, and Decapsulation: The most expensive mathematical operation in the scheme is the matrix-vector multiplication $\mathbf{A} \circ \mathbf{s}$, where $\mathbf{A}$ is a $k \times k$ matrix and $\mathbf{s}$ is a vector of length k . Because the scheme uses the NTT domain, this matrix-vector multiplication requires k^2 polynomial multiplications. Since $n = 256$ is a fixed constant, the asymptotic time complexity scales at $O(k^2 \cdot n \log n)$ for Key Generation, Encapsulation, and Decapsulation.
 - Symmetric Primitives: A significant portion of the practical running time is also spent on symmetric cryptographic functions (e.g., generating the uniform matrix $\mathbf{A}$ using SHA3/SHAKE-128 and sampling noise vectors via SHAKE-256). The time spent hashing depends on the parameter k as the scheme must pseudo-randomly generate the $k \times k$ matrix entries.
- Practical Performance (Time): Because $n = 256$ and $q = 3329$ are fixed, ML-KEM is extremely fast in practice. According to benchmarks of the AVX2-optimized implementation running on an Intel Core i7-4770K (Haswell) processor, the exact CPU cycle counts scale directly with k :

Variant	KeyGen	Encaps	Decaps
ML-KEM-512 ($k = 2$)	~33,856 cycles	~45,200 cycles	~34,572 cycles
ML-KEM-768 ($k = 3$)	~52,732 cycles	~67,624 cycles	~53,156 cycles
ML-KEM-1024 ($k = 4$)	~73,544 cycles	~97,324 cycles	~79,128 cycles

2. Space Complexity:

- The space complexity scales linearly with the module dimension k and the specific number of bits dropped during the compression function (used to reduce ciphertext sizes). The NIST FIPS 203 specification outlines the exact space requirements (in bytes) for the three security parameter sets

Parameter Set	Security Level	Encapsulation Key (Public Key)	Decapsulation Key (Private Key)	Ciphertext	Shared Secret Key
ML-KEM-512 ($k = 2$)	Category 1 (AES-128)	800 bytes	1632 bytes	768 bytes	32 bytes
ML-KEM-768 ($k = 3$)	Category 3 (AES-192)	1184 bytes	2400 bytes	1088 bytes	32 bytes
ML-KEM-1024 ($k = 4$)	Category 5 (AES-256)	1568 bytes	3168 bytes	1568 bytes	32 bytes

- Memory Allocation Limits: Because ML-KEM objects are small, conforming implementations require very modest memory overhead. The authors note that the algorithms do not require dynamic memory allocation on the heap and require a minimal stack size (e.g., standard optimized ARM Cortex-M4 implementations use less than 4 Kilobytes of RAM overall).

3. Algorithm:

- To understand where the $O(k^2 \cdot n \log n)$ time complexity comes from, it helps to look at the mathematical machinery inside ML-KEM. At its core, ML-KEM is based on the Module Learning with Errors (MLWE) problem.
- The Core Mathematical Objects: Before diving into the steps, we need to define the structures the algorithm operates on:
 - Polynomials: Instead of standard integers, ML-KEM operates on polynomials of degree up to 255 (meaning $n = 256$). The coefficients of these polynomials are integers modulo a small prime ($q = 3329$).

- Vectors and Matrices: The algorithm scales its security by grouping these polynomials into arrays (vectors) of length k and matrices of size $k \times k$. For example, ML-KEM-768 uses a 3×3 matrix ($k = 3$).
- Step-by-Step Algorithm: ML-KEM is built using a public-key encryption scheme (called K-PKE) as a subroutine. The KEM process consists of three main phases:
 - Phase 1: Key Generation (Alice creates her keys):
 - Generate a Matrix: Alice generates a random $k \times k$ matrix $\hat{\mathbf{A}}$ consisting of polynomials.
 - Generate Secrets and Errors: Alice samples a short secret vector $\mathbf{s}$ and a small error (noise) vector $\mathbf{e}$, both of length k .
 - Compute the Public Key: Alice calculates her public key vector $\hat{\mathbf{t}}$ using the formula: $\hat{\mathbf{t}} = \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$.

Where does the complexity come from? Multiplying the $k \times k$ matrix $\hat{\mathbf{A}}$ by the length- k vector $\hat{\mathbf{s}}$ requires calculating k^2 polynomial multiplications.

- Phase 2: Encapsulation (Bob creates a shared secret and ciphertext):
 - Generate Ephemeral Secrets: Bob wants to securely send a secret to Alice. He generates a random ephemeral secret vector $\mathbf{y}$ and his own error vectors $\mathbf{e}_1$ and $\mathbf{e}_2$.
 - Compute Ciphertext: Bob computes two components to form the ciphertext:
 $\mathbf{u} = \hat{\mathbf{A}}^T \circ \mathbf{y} + \mathbf{e}_1$ and $v = \hat{\mathbf{t}}^T \circ \mathbf{y} + \mathbf{e}_2 + \text{message}$

Where does the complexity come from? Bob must transpose the $k \times k$ matrix $\hat{\mathbf{A}}$ and multiply it by his length- k vector $\mathbf{y}$. Again, this takes k^2 polynomial multiplications.

- Phase 3: Decapsulation (Alice recovers the shared secret)
 - Decrypt the Message: Alice uses her secret vector $\mathbf{s}$ to recover Bob's message by computing $v - \hat{\mathbf{s}}^T \circ \mathbf{u}$. (The errors cancel out due to mathematical rounding).
 - Re-encryption (The "CCA Transform"): To ensure the ciphertext wasn't tampered with (achieving active security), Alice takes the recovered message and completely re-runs Bob's Encapsulation step to see if the resulting ciphertext matches the one Bob sent her.

Where does the complexity come from? Re-running the encapsulation step means Alice must perform the $\hat{\mathbf{A}}^T \circ \mathbf{y}$ matrix-vector multiplication again, taking k^2 polynomial multiplications.

- Tying it to $O(k^2 \cdot n \log n)$:
 - The heaviest lifting in every single phase of the algorithm is computing the matrix-vector multiplication ($\mathbf{A} \circ \mathbf{s}$ or $\mathbf{A}^T \circ \mathbf{y}$). Because the matrix is $k \times k$ and the vector is length k , computing the result requires exactly k^2 polynomial multiplications.
 - Now, how hard is a single polynomial multiplication?
 - Normally, multiplying two polynomials of degree n takes $O(n^2)$ operations. For $n = 256$, that would be $\approx 65,536$ operations per multiplication, which is far too slow.
 - To fix this, ML-KEM uses the Number-Theoretic Transform (NTT). The NTT is a specialized version of the Fast Fourier Transform (FFT).
 - By converting the polynomials into the "NTT domain" before multiplying them, the algorithm can multiply them coefficient-by-coefficient. The NTT algorithm itself takes $O(n \log n)$ operations.